



Nombre y Apellido: VALENTIN ROSSO

Legajo:

Final

1. Se representan secuencias mediante árboles binarios dados por el siguiente tipo de datos:

`data Tree a = E | L a | N Int (Tree a) (Tree a)`

donde el recorrido *inorder* del árbol da el orden de los elementos de la secuencia y el valor entero guardado en los nodos representa la longitud de la misma.

Definir en Haskell, de manera eficiente, la función `isPrefixOf :: Eq a => Tree a -> Tree a -> Bool`, que determine una secuencia es prefijo de otra.

Por ejemplo,

`isPrefixOf ('a', 'b') ('a', 'b', 'c') = True`

`isPrefixOf ('a', 'b') ('c', 'a', 'b') = False`

Ayuda: Puede ser de utilidad definir una función auxiliar que retorne además del booleano la secuencia que sobra al sacarle el prefijo. Por ejemplo,

`auxiliar ('a', 'b') ('a', 'b', 'c', 'd') = (True, ('c', 'd'))`

En teoría de grafos un ciclo de Hamilton es un ciclo que recorre cada vértice del grafo exactamente una vez (salvo el primer vértice que coincide con el último).

Suponer que los grafos están representados como tablas de adyacencias, es decir que $Graph\ V = Table\ V\ S$. Definir las siguientes funciones:

a) `esCamino :: Graph V -> Seq V -> Bool`, que determine si una secuencia de vértices es un camino simple en el grafo. Un camino es simple en el grafo si no se repiten vértices.

b) `esCicloSimple :: Graph V -> Seq V -> Bool`, que determine si una secuencia de vértices es un ciclo simple en el grafo. Un ciclo es simple en el grafo si no se repiten vértices, salvo el primero y el último.

c) `esCicloHamilton :: Graph V -> Seq V -> Bool`, que determine si una secuencia de vértices es un ciclo Hamilton en el grafo.